eCAT User Guide

electroChemical Analysis Tools | Internal lab beta v0.1.0b5

Table of Contents

Quick orientation

eCAT helps electrochemists load, inspect, plot, filter, analyze, and export electrochemical data from common lab workflows. Most users should start with get_data() or get_CVs(), inspect the returned objects, plot them, then run only the analysis functions that match the experiment and beta scope.

Installation

Install eCAT in a clean Python environment, then import it as e. If Matplotlib reports a cache-directory warning on first import, set MPLCONFIGDIR to a writable folder or continue after the cache is built.

eCAT app

The eCAT app uses optional GUI/web dependencies. Install them with python -m pip install "ecat[app]"; for a source checkout, use python -m pip install -e ".[app]". Terminal users can open the native app window with:

```
ecat-app
```

Notebook users can launch the same native app from Python:

```
import ecat as e
e.open_app()
```

Use ecat-app --browser or e.open_app(browser=True) if you prefer a browser tab. Use e.open_app(inline=True) to display browser mode in a notebook iframe when the notebook environment supports it. Browser mode opens at http://127.0.0.1:8050 by default and automatically uses the next available port if 8050 is busy; native-window mode uses a private available port by default.

Five-minute usage path

1. import ecat as e
2. Load files with e.get_data({'folder path': path}) or e.get_CVs({'folder path': path})
3. Inspect objects with e.show_objects(objects), obj.info(), obj.stats(), obj.x(), obj.y(), or obj.data
4. Plot one object with obj.plot() or several with e.multiplot(objects)
5. Run a focused analysis such as cv_obj.peak_current(...) or e.fowa(cvs, ...)
6. Export processed data with e.save_data(objects, ...)

Function and class index

Core objects: echem, cv, ca, cp, dpv.

Loading: echem.from_file, parse_file, get_data, get_CVs, get_data_from_excel.

Inspection: info, stats, x, y, xy, show_objects.

Plotting: plot, animate, multiplot, multimultiplot, multi_scatterplot, plotting_style. Scale bars are provided through the scale bar plot option.

Filtering and grouping: filter, sort, group, sort_and_group, group_summary, get_available_filter_values.

CV analysis: peak_potential, peak_current, peak_width, half_peak_potential, half_wave_potential, current_at_potential, normalize, normalize_current, scale_current.

Advanced analysis: fowa, sevcik_analysis, trumpet_analysis, nicholson_analysis, tafel_analysis, fit_model, fit_rate, plateau_current, fit_peak_potential, fit_peak_current.

Export and defaults: save_data, describe_options, get_defaults, set_defaults, reset_defaults, load_defaults.

Simulation namespace: supported simulation helpers live under ecat.simulation because they require explicit mechanism/parameter inputs and, for numerical simulation, the optional ElectroKitty backend.

Loading data

Parser contract: every loaded object exposes obj.parse_result, a ParseResult with data, units, technique, software, metadata, raw_metadata, warnings, source, and parser. Use e.parse_file(path, options) when you want that parser contract directly for importer debugging or tests; use echem.from_file(), get_data(), or get_CVs() for ordinary analysis workflows.

Current text parser coverage: EC-Lab-style text now has tested ParseResult-first coverage for CV, CA, and CP/GCPL-like imports; NOVA ASCII CV text has limited coverage; and generic potential/current text can load as a CV-like fallback with parser warnings. Ambiguous generic tables containing time, potential, and current columns stay generic unless a vendor or user-supplied technique marker resolves the technique. CH and BASI keep their established reader paths for beta stability. IviumSoft text and DPV text beyond existing CH/private-fixture coverage still need representative files before they should be treated as beta-supported workflows.

Use get_data() for mixed supported electrochemistry folders, get_CVs() for flexible CV text-table workflows, and get_data_from_excel() for eCAT Excel workbooks or curated spreadsheet-assembled CV datasets. Binary files are not beta-ready unless explicitly documented elsewhere.

Data collection and naming

Recommended names make loading, filtering, grouping, plotting, and simulation fitting much smoother because they give eCAT reliable metadata even when instrument exports are sparse.

- Put technique, solvent, gas/atmosphere, scan rate or window, compounds, concentrations, and run/replicate identifiers in the file or folder name when those values may not survive the instrument export.
- Keep each concentration adjacent to its compound name when possible, such as 100mMPhOH, 1mMCo, 3mMFc, or 5%CO2.
- Use explicit units: M, mM, uM, µM, nM, %, equiv, and x are recognized. Avoid bare lowercase m for molarity; write 100mM, not 100m.
- Choose species names that match later analysis or simulation names when practical. If experimental names differ from simulation species, record the mapping in the notebook with options such as concentration mapping.
- Good compact examples: 100mVs_CO2_MeCN_1mMFe-tpyPY2Me_100mMPhOH_run01 or CV_MeCN_Ar_0.1MTBAPF6_3mMFc_1mMFe_100mVs.

Plotting and grouping

Use obj.plot() for one object, animate() or obj.animate() for notebook-friendly playback, and multiplot() for overlays. animate() auto-displays in notebooks when 'plot' is True, suppresses the empty static placeholder figure, and returns an AnimationResult for reuse or saving. Use plotting_style("notebook"), plotting_style("publication"), or plotting_style(False) to control eCAT plotting defaults. Use filter(), sort(), group(), and sort_and_group() to organize loaded objects before plotting. The package default plot convention is IUPAC; for CVs, use {'plot convention': 'US'} when you want the historical US axis orientation instead.

Use the plot option symbol labels to switch axis labels from descriptive text to compact electrochemistry symbols. With symbol labels enabled, potential, current, current density, time, and charge are labeled E, i, j, t, and Q. The Savéant plotting style enables symbol labels automatically; notebook and publication styles keep descriptive labels unless symbol labels is set to True. Normalized axes such as i/ip0 remain symbolic.

CV analysis

CV-specific analysis includes peak potentials, peak currents, half-wave estimates, current normalization, dimensionless normalization, and current scaling. Top-level normalize() is CV-only and returns copies; cv.normalize() mutates one CV object.

CA/CP basics and limitations

For beta, CA and CP support includes load, inspect, plot, export, and tested technique-specific basics. CA and CP show() summaries include the same basic filename and condition metadata as CVs, including solvent, gas, compounds, and concentrations when those values were parsed or supplied. Compact filename concentration parsing requires explicit units such as M, mM, uM, or µM; typo-like bare lowercase m tokens such as 100mTBAPF6 are ignored rather than treated as concentrations. Standalone reference-electrode salt decorations such as AgAgBF4-5mM are not treated as sample analyte concentrations. CA objects expose charge(options=None) for cumulative charge in Coulombs; it returns a dict-like result with the raw charge trace, final charge, optional target charge, time at target charge, and plotted axes. Baseline cleanup is available with {'baseline correction': True}; eCAT uses {'baseline correction': 'threshold'} when {'baseline threshold': value_in_A} is supplied and otherwise uses {'baseline correction': 'tail'} with the final {'baseline tail fraction'} of the trace, defaulting to 0.05. Corrected charge values are returned alongside the raw charge values. time_at_charge() prints using the same auto-scaled time unit shown on plots unless an x unit is specified. CA and CP default plot titles use the same title/subtitle style as CV and DPV: the figure title emphasizes sample or compound identity, while the axes subtitle carries technique and conditions. CP objects expose cycle_info(), cycling_plot(), and plot_cycles(); cycling_plot() overlays capacity and efficiency versus cycle number, while plot_cycles() overlays selected potential/capacity or potential/time cycle traces. Both cycling_plot() and plot_cycles() accept {'cycles': int_or_list_or_tuple}; tuples may be (start, end) or (start, end, step). plot_cycles() can color selected cycles by cycle number with {'color mode': 'gradient'} and {'legend mode': 'colorbar'}. Default printed object lists omit IR compensation columns and CA min/max/average current columns unless those values are inspected through stats().

CA objects also expose current_at_time(), average_current(), rate_at_time(), and average_rate(). These helpers return dict-like ChronoAnalysisResult objects with a Metric/Value table and print the relevant interpolation, averaging, or rate equations by default. Rate helpers report electron flow from i/F and, when electron count, Faradaic efficiency, and catalyst amount are supplied, molecular rate and TOF.

Advanced analysis

Advanced workflows such as fowa, Sevcik-style analysis, trumpet analysis, Nicholson analysis, Tafel-style analysis, and fit helpers should be used with attention to assumptions, fit ranges, and warnings.

Advanced analysis return contract: complex analyses return AnalysisResult-style objects. Use .table for the primary table, .summary for workflow metadata, .fits or .fit_table where fitting applies, .axes for plots, .units for result units, and .warnings or .diagnostics for extra detail. Use result.to_csv(...) to export the primary table as CSV, or result.to_excel(...) to write an Excel workbook with table, summary, fit_table, fits, warnings, units, and diagnostics sheets when present.

Simulation namespace

Simulation and global-fit helpers are supported advanced workflows under ecat.simulation. Treat fitted mechanisms as scientific models: record assumptions, parameter bounds, corrections, and fit diagnostics.

Troubleshooting

When something fails, record the package version, function call, options, file type, expected behavior, actual behavior, and traceback. Common beta issues include unsupported file formats, unit confusion, reference-shift setup, normalization parameters, missing peaks, and parser failures.

For parser issues, inspect obj.parse_result.warnings and obj.parse_result.raw_metadata. Warnings are nonfatal diagnostics that often indicate missing or inferred metadata such as scan rate, timestamp, step structure, or technique. Use e.parse_file(path) to inspect the parser contract directly before object promotion.

Beta scope and limitations

The beta priority is trustworthy core workflows, clear limitations, and useful feedback from real lab data. Optional features should not block installation, loading, plotting, core CV analysis, export, and documentation.

Parser scope note: binary vendor files remain out of beta scope. Generic text import is a fallback, and IviumSoft or DPV text workflows should be validated with representative files before being used for beta feedback.

Detailed Function Reference

Notebook import Install the package into the same Python environment as the notebook kernel, then use import ecat as e. The compatibility alias import eCAT as e is also included.

```
# Install from a wheel in the active notebook kernel
%pip install r"C:/path/to/ecat-0.1.0b5-py3-none-any.whl"

import ecat as e
```

Options model Public functions still accept simple dictionaries. Internally, eCAT validates option names, values, choices, and routing before analysis starts. Use e.describe_options('function_name') to inspect the full schema from a notebook.

Common option helpers

Task	Notebook command	Notes
Inspect options	e.describe_options('multiplot')	Shows defaults, choices, and descriptions.
Set one default	e.set_defaults('fowa', {'fit basis': 'y'})	Section-specific defaults override package defaults.
Set a global default	e.set_defaults('print', False)	Applies to option models that support print.
Reset defaults	e.reset_defaults()	Restores package defaults for the session.

Extracting, Organizing, & Saving Data

get_data(options=None)

Description: Reads electrochemical text files, creates echem/cv/cp/ca/dpv objects, sorts them, and attaches reference-potential metadata when requested.

Parameters

- options (dict or ImportOptions): import, parser, metadata, sorting, and reference-shift controls.

Key options

Option	Default	Type	Choices	Description
folder path	.	str		Folder path searched for electrochemical data files.
recursive search	True	bool		Recursively search subfolders during import.

Option	Default	Type	Choices	Description
sort keys	subfolder, timestamp	object		Sort keys used to order imported objects after parsing and before reference assignment.
software		str or None		Instrument software or parser to use during import.
reference mode	auto	str	auto, manual, keyword, file, none	Reference mode for import reference shifting or scale_current reference-wave selection.
reference keyword		str or None		Single filename keyword used to identify reference files.
reference file		str or None		Explicit file used as a reference-shift source.
reference guess	auto	float or str or None		Potential guess used to locate a reference wave.
shift potential	False	bool or str or float		Legacy flag or value for shifting the potential axis.
electrode diameter	0	float		Electrode diameter in cm.
electrode area	0	float		Electrode area in cm^2.
current density	False	bool		Normalize current by electrode area to report current density.
invert current	False	bool		Multiply current by −1.
name alterations		object or None		Filename text replacements applied during import.
print	True	bool		Whether to print a result summary.
troubleshoot	False	bool		Print additional troubleshooting output.
reference map		dict or None		Explicit imported-object index mapping, e.g. {45: 54}, where the key is the object to shift and the value is the object used as its reference source.

Behavior notes

- 'reference map' uses the sorted/displayed object indices and overrides automatic self/folder reference assignment for the mapped targets.
- Default folder import order keeps subfolders together, then orders objects within each subfolder by acquisition timestamp when instrument timestamps are available. Timestamp sorting falls back to filesystem creation and modification times, with folder-relative path ordering used for stable discovery.
- Generic fallback objects parse gas and solvent from filenames. CH-style CPE/CA, CV, and DPV imports keep IR compensation metadata for sorting, grouping, and table/stat display when those fields are requested. Single-object show() displays IR compensated and uncompensated resistance values with ohm in the value column. For single-file loading, e.echem.from_file(path, {'print': True}) displays the loaded object with e.show().
- When 'reference mode' is 'file', 'reference file' may be absolute or relative to 'folder path'. eCAT loads that file, locates the reference couple using the same reference-guess and shift-window options, and applies the resulting potential shift to imported CVs with reference mode 'file'.

EC-Lab GCPL exports are imported as chronopotentiometry (cp) objects when the technique line is Galvanostatic Cycling with Potential Limitation. Numeric time columns and absolute timestamp columns are converted to elapsed seconds, Ewe

is stored as Potential in V, and programmed Is step currents are captured as anodic_current and cathodic_current when present.

- If an individual .txt file cannot be converted, get_data() prints a warning with the filename and error, skips that file, and returns the successfully imported objects. If no files can be converted, it prints a warning and returns an empty list [].

Returns: list[echem]. If no files convert successfully, returns an empty list [] after printing a warning.

Example

```
objects = e.get_data({
  'folder path': r'C:/data/CV_runs',
  'recursive search': True,
  'reference mode': 'keyword',
  'reference keyword': 'Fc',
  'reference map': {45: 54},
})
```

- Filename metadata recognizes mole-fraction tokens with the unit x, such as 0.8xD2O, 0xD2O, and 1xD2O. Endpoint mole fractions must be explicit; missing x tokens are not inferred as 0 or 1. User-facing labels display mole fractions as χ(species), for example χ(D2O).

Filename parsing and recommended names

eCAT uses filename parsing as a practical metadata fallback when a file does not provide gas, solvent, compound/concentration, or scan-rate metadata clearly.

- Built-in filename parsing looks for gas, solvent, compounds, concentrations, and scan rate.
- Recognized concentration units in filenames include M, mM, uM, µM, nM, L, %, equiv, and x.
- Recognized scan-rate patterns include compact and spaced forms such as 100mVs, 100 mV/s, 0.1Vs, and 500uV/s.
- Recognized pressure units for metadata and display include Pa, bar, atm, Torr, mmHg, and psi. SI prefixes are supported where they are natural for pressure, such as kPa, MPa, and mbar. eCAT preserves pressure units by default and converts between pressure units only when a target/display unit is requested explicitly.
- Recommended filename style is explicit and compact, for example 100mVs_CO2_MeCN_1mMFe-tpyPY2Me_100mMPhOH_run01 or CV_MeCN_Ar_0.1MTBAPF6_3mMFc_1mMFe_100mVs.
- Keep each concentration adjacent to its species when possible, for example 100mMPhOH. Human-readable space-delimited names are supported only as a fallback.
- Bare lowercase m is intentionally not treated as molar because it is too ambiguous. If the built-in parser is not enough, use the compounds option, a custom parser, or parser settings.
- File-derived metadata still wins by default. Use parser settings with prefer file metadata = False only when you explicitly want custom parser results to replace file metadata such as scan rate.

get_CVs(options=None)

Description: Convenience loader for cyclic-voltammetry tables. It accepts mixed delimiters, decimal-comma data, CH-style metadata, and simple Potential/Current files.

Parameters

- options (dict or ImportOptions): same import and metadata controls as get_data().

Key options

Option	Default	Type	Choices	Description
folder path	.	str		Folder path searched for electrochemical data files.

Option	Default	Type	Choices	Description
recursive search	True	bool		Recursively search subfolders during import.
scan rate				Manual scan rate in V/s.
name alterations		object or None		Filename text replacements applied during import.
gas		str or None		Gas condition metadata for the electrochemical object.
solvent		str or None		Solvent metadata for the electrochemical object.
compounds		object or None		Compound names associated with the electrochemical object.
convert current	False	bool		Convert current values to a requested display or analysis unit.
electrode area	0	float		Electrode area in cm^2.
print	True	bool		Whether to print a result summary.

Returns: list[cv] or None.

Example

```
cvs = e.get_CVs({'folder path': r'C:/data/CVIS', 'scan rate': 0.05})
```

filter(object_list, filter_keys, options=None)

Description: Filters flat or grouped echem objects by metadata such as scan rate, gas, solvent, species, scan window, replicate, and chrono keys such as applied potential, run time, final charge, cathodic current, anodic current, and quiet time.

Parameters

- object_list: echem objects or nested groups.
- filter_keys: requested metadata values.
- options (dict or FilterOptions): filtering and printing controls.

Key options

Option	Default	Type	Choices	Description
print	True	bool		Whether to print a result summary.
pretty print	True	bool		Use rich table display when printing object lists or summaries.
print conditions	True	bool		Whether to include condition columns in printed object tables.
mode	include	str	include, exclude	Mode selector for the current operation.
logic		str or None	AND, OR	Logical rule used to combine filter criteria.

Returns: Filtered list, preserving group structure when groups are provided.

Example

```
fc = e.filter(objects, {'gas': 'CO2', 'species': '100 mM Fc'}, {'mode': 'include'})
```

sort_and_group(object_list, sort_keys=None, group_keys=None, options={'print': True})

Description: Sorts echem objects by selected metadata keys, then groups objects sharing selected grouping keys. Chrono objects can be sorted/grouped by SI-valued keys such as applied potential, sample interval, run time, final charge, cathodic current, anodic current, high potential limit, and low potential limit.

Parameters

- sort_keys: one key or a list such as ['gas', 'compounds', 'timestamp'], ['applied potential'], or ['final charge'].
- DPV analysis helpers include dpv.peak_potential(options=None) for locating a single DPV extremum and dpv.fit_overlapping_peaks(options=None) / dpv.closely_spaced_peaks(options=None) for fitting a linear baseline plus two Gaussian peak components. Use guess potentials to seed closely spaced peaks; cathodic peak amplitudes are negative unless current is inverted. For difficult overlaps, use fit window to limit the optimizer to the feature region, center window to keep each peak center near its guess, and sigma guess / sigma min / sigma max to tune peak widths. When plot=True, fit_overlapping_peaks() overlays the total fit and the individual Gaussian peak components; the overlay legend is shown by default with labels DPV Data, Total Fit, Peak 1, and Peak 2. Customize labels with data label, fit label, peak 1 label, and peak 2 label, or hide the overlay legend with {"legend": False}. DPV pulse metadata follows the CA/CP unit convention: attributes and stats keys are unitless SI values, while printed summaries and plot subtitles put autoscaled units in the displayed values, such as 10 mV or 50 ms.
- options (dict or SortGroupOptions): display controls.

Key options

Option	Default	Type	Choices	Description
print	True	bool		Whether to print a result summary.
pretty print	True	bool		Use rich table display when printing object lists or summaries.
print conditions	True	bool		Whether to include condition columns in printed object tables.

Returns: list[list[echem]] when grouping keys are supplied; otherwise a sorted list.

Example

```
groups = e.sort_and_group(objects, sort_keys=['gas', 'compounds', 'timestamp'], group_keys=['type'])
```

group_summary(object_list, group_keys=None, options=None)

Description: Summarizes a flat list of echem objects or an existing grouped list, returning one pandas DataFrame row per group.

- object_list: flat echem object list or nested groups produced by group() / sort_and_group().
- group_keys: optional key or list of keys. When provided, it overrides options['group keys'] and groups internally using group(..., {'print': False}).
- options (dict or GroupSummaryOptions): 'group keys', 'columns', 'print', 'pretty print', and 'sig figs'.
- The summary includes group key values, total object count, CV/CA/CP/DPV counts when present, and distinct counts plus value lists for requested or varying metadata columns.
- Concentration grouping follows group() exactly, including grouping by the full parsed concentrations list.

Returns: pandas.DataFrame with display-ready summary columns.

```
summary = e.group_summary(objects, group_keys='concentrations', options={'columns': ['scan rate']})
```

save_data(object_list, options=None)

Description: Exports echem object data to CSV or to an eCAT Excel workbook. CSV is the flat external table format. Excel uses a manifest sheet plus one sheet per object class for human-editable, round-trip-friendly data. Referenced CVs export both the stored Potential axis and the active Potential vs ... axis by default when a reference shift is stored.

Parameters

- object_list: echem objects or groups.
- options: output folder, filename, format, metadata columns, data columns, share x axes, and unit conversion controls. Use {'format': 'xlsx'} for Excel export. metadata columns is additive like show(); data columns is strict and can be 'all', 'stored', or an exact list of exported data columns.

Returns: pd.DataFrame for CSV export, or a dict of workbook DataFrames for Excel export.

Example

```
df = e.save_data(groups, {'folder path': r'C:/data/processed', 'file name': 'grouped_cv_data'})
workbook = e.save_data(objects, {'format': 'xlsx', 'folder path': r'C:/data/processed', 'file name': 'experiment_export', 'metadata columns': ['reference source'], 'data columns': 'all', 'share x axes': True})
```

get_data_from_excel(file_path, options=None)

Description: Imports eCAT Excel workbooks when a manifest sheet is present. If no manifest exists, falls back to curated single-workbook CV header parsing using curve names in the first header row, axis labels in the second header row, and the same filename/header metadata parser settings as other eCAT imports.

Parameters

- file_path: Excel workbook path.
- options: import/object construction controls.

Key options

Option	Default	Type	Choices	Description
print	True	bool		Whether to print a result summary.
troubleshoot	False	bool		Print additional troubleshooting output.
name alterations		object or None		Filename text replacements applied during import.
electrode area	0	float		Electrode area in cm^2.
electrode diameter	0	float		Electrode diameter in cm.

Use get_data_from_excel() for Excel import. Manifest workbooks can contain CV, CA, CP, and DPV objects; fallback header parsing returns CV objects.

Returns: list of eCAT objects. Manifest workbooks can include CV, CA, CP, and DPV objects; fallback header parsing returns CV objects.

Example

```
objects = e.get_data_from_excel('processed_curves.xlsx', {'print': True})
```

Plotting

echem_object.plot(options=None, **mpl_kwargs)

Description: Plots one echem object using its selected x/y data. CVs can plot selected segments; CA current plots include 0 on the y axis by default, and CA/CP plots use auto titles consistent with CV/DPV. CA objects can add cumulative charge on a secondary axis with {'plot charge': True}. Use {'charge color': 'tab:green'} or another Matplotlib color to color the secondary charge line, right-axis label, ticks, and spine together. ca.charge() plots cumulative charge in black unless a color is supplied, accepts regular plot options such as title, and can plot baseline-corrected charge when baseline correction options are supplied. CP cycling_plot() overlays capacity/efficiency performance metrics and accepts {'cycles': ...} to select points by cycle number. CP plot_cycles() shows selected cycle traces and supports gradient cycle coloring with the same custom colorbar style used by multiplot.

Parameters

- options (dict or PlotOptions): axis, unit, segment, minor-tick, and styling controls.
- Use 'derivative': 1 or 2 to plot Savitzky-Golay derivative views; derivative is a plotting view and does not mutate the stored current data. Current-vs-potential derivative plots label the y axis as di/dE or d^2i/dE^2 with composed units such as microampere per volt.
- For current-density views, set 'y axis': 'current density' with an electrode area. eCAT reports the plot-time axis as A/cm^2 by default and auto-scales the numerator for readable values, such as microampere per cm^2. The 'y unit' option accepts current-only units such as 'uA', area-only shorthand such as 'mm', 'mm^2', or '/mm^2', and full units such as 'uA/mm^2'. Full current-density units are used exactly, while area-only shorthand keeps numerator auto-scaling.
- CV segment coloring: 'segment color mode' defaults to 'auto', which uses grouped discrete-gradient colors when a CV has more than 3 plotted segments; set it to 'off', 'discrete', 'discrete gradient', or 'continuous gradient' to choose explicitly.

cv.plot() opens a new figure by default so repeated notebook calls do not silently overlay traces; set 'new plot': False when you intentionally want to add a CV to the current axes. eCAT plot styling is active on import with the notebook profile, which uses a compact VS Code/Jupyter-friendly figure size and typography, one minor tick between adjacent major ticks, and capped automatic legend font sizes. Call e.use_ecat_plot_style('publication') for larger export-oriented figures, e.use_ecat_plot_style('notebook') to switch back, or e.use_ecat_plot_style(False) to restore Matplotlib defaults.

For CV potential programs, cv.plot_program(options=None) plots potential versus time. If the CV has a time column, eCAT uses it; otherwise it reconstructs elapsed time from scan rate. Quiet time is included by default as a negative-time hold at the starting potential; pass {'plot quiet time': False} to omit it.

Use cv.trim({'potential window': [start, stop]}) to return a copied CV object trimmed to a displayed potential window before plotting or downstream analysis. cv.trim accepts the usual eCAT options-dict style or TrimOptions; convenience forms such as cv.trim([start, stop], window_mode='allow') are also supported. The default window mode is 'expand', which preserves a connected CV waveform when a pointwise window would disconnect the scan history; use 'allow' for pointwise trimming or 'strict' to reject disconnected windows. The original CV is unchanged unless inplace=True is supplied.

- Use 'minor ticks' to control per-axis minor ticks: True uses the eCAT default, False disables minor ticks, and an integer sets the AutoMinorLocator subdivision count.
- Use 'segment color groups' to group segment colors; an integer is a minimum group size, while explicit groups like [[1, 2], [3, 4, 5]] preserve original segment numbers in labels. Segment colorbars use the same custom inset legend style and adaptive placement as multiplot, with 'Segments' as the context line and numeric segment labels on the bar; 'legend': 'auto' only displays a legend/colorbar when there is more than one entry, and 'legend loc': 'auto' chooses a low-overlap location. Set 'colorbar style' to 'auto', 'discrete', or 'continuous'; 'auto' follows the segment color mode, so 'discrete gradient' uses a continuous colorbar while 'discrete' forces swatches.

- **mpl_kwargs: passed through to matplotlib plotting where supported.

Key options

Option	Default	Type	Choices	Description
x axis		str or None		Column or axis used as x data.
y axis		str or None		Column or axis used as y data.
x unit	auto	str or None		Requested x-axis unit.
y unit	auto	str or None		Requested y-axis unit.
plot convention	US	str	US, IUPAC	Electrochemical plotting convention for axis orientation and signs.
plot segment		int or None		Segment to emphasize or plot.
plot segments		list[int] or int or None		Segments to emphasize or plot.
color	black	str or None		Primary plot color.
label		str or None		Label for a plotted trace or analysis output.
legend	auto	bool or str		Whether to show a plot legend.
title	True	bool or str		Plot title text or title mode.
new plot	False	bool		Create a new figure before plotting.
ip0				Non-catalytic reference peak current.
non catalytic current				Manual non-catalytic reference current.

Returns: matplotlib.axes.Axes for single-panel plots. Access the figure as ax.figure.

Example

```
cv.plot({'plot segments': [1, 2], 'y axis': 'i/ip0', 'ip0': 2e-6, 'color': 'tab:blue'})
cv.plot({'segment color mode': 'discrete gradient', 'segment color groups': 2})
```

multiplot(echem_list, options=None)

Description: Plots many echem objects on one figure with shared unit scaling, automatic labels, optional gradients, and adaptive legend/colorbar behavior.

Parameters

- echem_list: echem objects to overlay.
- Set 'legend mode': 'discrete' with 'legend outside': True to place a normal trace legend outside the axes.
- Use 'min gradient entries' to choose how many distinct gradient values are required before auto legend mode uses gradient colors and a colorbar; duplicate traces at the same value do not increase the count, and smaller groups remain discrete. This option also routes through FOWA plot-all multiplots.

Use 'deduplicate labels' to distinguish repeated multiplot labels after label alterations. The default False leaves labels unchanged but warns when duplicates are detected and suggests {'deduplicate labels': True}. Set it to True to append only

differing scan window and segment metadata, omitting fields that match; if labels still collide, eCAT appends replicate numbering. Pass a valid metadata key or list of keys to choose the metadata used; invalid keys print the valid options.

- Mole-fraction colorbars use χ(species) endpoint labels and do not add the leading '+' used for concentration additions.
- Gradient-colored multiplots separate color mapping from colorbar display: 'gradient ...' options choose how trace metadata maps to colors, while 'colorbar ...' options control the displayed colorbar legend. Use 'colorbar tick labels' ('endpoints', 'all', or 'none') and 'colorbar trace ticks' to control tick text and per-trace tick marks. With 'gradient scale': 'auto', scan-rate gradients and positive concentration gradients use log spacing; percent and equivalent concentration series follow that same concentration-gradient default, while zero-concentration entries are treated as background traces rather than zero-valued colorbar endpoints. 'gradient reverse' reverses trace color assignment; 'colorbar reverse' flips only the displayed colorbar direction.
- options (dict or MultiplotOptions): shared axis, label, color, legend, and layout controls.

Key options

Option	Default	Type	Choices	Description
x axis				Column or axis used as x data.
y axis				Column or axis used as y data.
x unit				Requested x-axis unit.
y unit				Requested y-axis unit.
labels		list[str] or None		Explicit labels for plotted traces; plot labels is accepted as a legacy alias.
label alterations				Text replacements applied to generated labels.
title	auto	bool or str		Plot title text or title mode.
subtitle	auto	bool or str or None		Plot subtitle text or subtitle mode.
color mode	auto	str	auto, gradient, discrete	Color assignment mode for multi-trace plots.
legend mode	auto	str	auto, colorbar, discrete	Legend mode for discrete or gradient color encodings.
legend loc	auto	str		Matplotlib legend location.
legend outside	False	bool		Place the legend outside the axes.
colorbar height scale	1.0	float		Scale factor for colorbar height in gradient legends.
gradient by	auto	str	auto, scan rate, concentration	Metadata field used to assign gradient colors.
gradient species	auto	str		Species used to resolve concentration-based gradient coloring.
gradient scale	auto	str	auto, linear, sqrt, log, index	Scale used to map gradient values into the colormap.
offset				Vertical offset applied to plotted traces.

Option	Default	Type	Choices	Description
stacking				Vertically stack plotted traces.

Returns: matplotlib.axes.Axes for the shared multiplot panel.

Example

e.multiplot(cvs, {'y axis': 'i/ip0', 'ip0': [1e-6, 1.2e-6], 'legend mode': 'discrete', 'legend outside': True})

multimultiplot(echem_groups, options=None)

Description: Creates one multiplot per group of echem objects.

Parameters

- echem_groups: list of groups or a flat list.
- options (dict or MultiMultiplotOptions): group titles/subtitles plus multiplot controls.

Key options

Option	Default	Type	Choices	Description
titles	auto	list[str] or str or None		Titles for multiple plots or panels.
subtitles	auto	list[str] or str or None		Subtitles for multiple plots or panels.
legend	True	bool or str		Whether to show a plot legend.
legend mode			auto, colorbar, discrete	Legend mode for discrete or gradient color encodings.
labels				Explicit labels for plotted traces; plot labels is accepted as a legacy alias.
color mode			auto, gradient, discrete	Color assignment mode for multi-trace plots.
gradient by			auto, scan rate, concentration	Metadata field used to assign gradient colors.
y axis				Column or axis used as y data.
y unit				Requested y-axis unit.

Returns: list of multiplot outputs.

Example

e.multimultiplot(groups, {'titles': 'auto', 'legend outside': True})

multi_scatterplot(datasets, options=None)

Description: Overlays labeled rate, peak-current, or peak-potential result tables. AnalysisResult or ScatterFitResult inputs reuse existing fit metadata for fit-line overlays.

Parameters

- datasets (dict): labels mapped to DataFrames or AnalysisResult objects.
- options (dict or MultiScatterplotOptions): column, style, legend, color, title, and fit-line display controls.
- Column selection: 'x column', 'y column', and 'y columns' select from existing result-table columns; they do not recompute upstream fits. Stored fit overlays are reused only when compatible with the selected columns. For example, plotting raw kobs from a fit_rate result that used a non-raw y mode requires a matching upstream fit_rate(..., {'y mode': 'raw'}) result or 'plot fit': False.

Key options

Option	Default	Type	Choices	Description
x column	auto	str or int or None		DataFrame column used as x data.
y column	auto	str or int or None		DataFrame column used as y data.
y columns		object or None		Multiple DataFrame columns used as y data.
metric		str or None		Metric column or quantity to analyze.
plot style	scatter	str	scatter, line, line+markers	Plot style such as scatter or line.
plot fit	True	bool		Whether to overlay fitted curves or lines.
legend				Whether to show a plot legend.
labels				Explicit labels for plotted traces; plot labels is accepted as a legacy alias.
label alterations				Text replacements applied to generated labels.
title				Plot title text or title mode.
subtitle				Plot subtitle text or subtitle mode.
fit linestyle	--	str		Line style for plotted fit lines.
fit linewidth	1	int or float		Line width for plotted fit lines.
fit alpha	1	int or float		Alpha transparency for plotted fit lines.

Returns: ScatterFitResult, an AnalysisResult child, with figure, axes, table, fit_table, and summary metadata.

- By default, multi_scatterplot displays the reused fit_table, including slopes and intercepts, when AnalysisResult inputs provide reusable fits. Set 'print': False to suppress this display.

Example

```
summary = e.multi_scatterplot({'Fe only': rate_result, 'cyclen': other_result}, {'legend': True})
```

- Use options['x column'], options['y column'], and options['y columns'] to choose exactly which existing result columns are plotted. multi_scatterplot does not reinterpret upstream analysis modes; create raw, transformed, or adjusted results upstream, then select the desired columns here.
- Transformed y-axis labels preserve stored y mode information, so fractional or adjusted results display labels such as log10(kobs/kobs0 - 1) instead of only log10(kobs).

- When raw or adjusted data are plotted from transformed fit results, built-in transforms such as log10, ln, reciprocal, and powers are back-transformed for fit overlays when possible.

animate(obj_or_list, options=None)

Description: Animates one electrochemical object or a list of objects using the same plot and multiplot styling that eCAT uses for fully rendered static figures.

Parameters

- obj_or_list: one echem/cv/ca/cp/dpv object or a list of objects.
- options (dict or PlotOptions): regular plot styling plus animation controls such as trace mode, schedule, timing mode, normalized duration, speedup, fps, stride, stagger time, end hold, cycle, include quiet time, print, pretty print, and progress. Use stride to render every nth plotted point for lighter animation previews or exports without changing the source object data.

Behavior notes

- Default animation behavior is object-aware: a single object uses trace mode = 'draw'; multiple objects use simultaneous draw playback when scan rates differ and staggered instant playback when scan rates match.
- Timing mode = 'auto' uses physical experiment timing when enough metadata are available; otherwise it falls back to normalized timing. In normalized mode, 'normalized duration' is the per-trace display time. In physical mode, 'speedup' compresses experiment time by the requested factor. Quiet time is excluded unless 'include quiet time' is True.
- Animation uses the same eCAT plot helpers as static plot()/multiplot() calls, so the fully rendered frame matches the corresponding static figure style, units, labels, and legend/colorbar behavior.
- Use result.save('movie.gif') or result.save('movie.mp4') to export the animation. GIF uses Pillow; MP4 uses Matplotlib's ffmpeg writer when available in the environment. Rendering progress is shown by default during display and export; set 'progress': False to suppress it.

Returns: AnimationResult
with .animation, .figure, .axes, .summary, .display(options=None), .show(options=None), .to_html(options=None), and .save(path, format=None, options=None).

Example

```
result = e.animate(cvs, {
    'gradient by': 'scan rate',
    'legend mode': 'colorbar',
    'timing mode': 'normalized',
    'normalized duration': 2.0,
    'fps': 20,
    'stride': 2,
    'progress': True,
})
result.save('scan_series.gif')
```

Individual CV Data Analysis

cv.peak_potential(options=None)

Description: Finds a peak potential on the selected CV segment using an exact potential, guess potential, or automatic extrema detection.

Parameters

- options (dict or PeakPotentialOptions): segment, smoothing, peak-selection, plotting, and printing controls.

Key options

Option	Default	Type	Choices	Description
segment				CV segment to analyze.
segments				One or more CV segments to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
peak prominence				Minimum peak prominence for automatic peak detection.
noise window				Savitzky-Golay smoothing window.
noise polyorder				Savitzky-Golay smoothing polynomial order.
plot				Whether to plot the result.
print				Whether to print a result summary.
plot all				Whether to show diagnostic or child plots.
troubleshoot				Print additional troubleshooting output.

When overlaying peak-current diagnostics on an existing cv.plot() or multiplot() axes, set {'plot': True, 'plot CV': False, 'new plot': False}. This draws the peak marker, tangent baseline, and current-distance line without redrawing the underlying CV trace.

Returns: CVAnalysisResult, an AnalysisResult child that behaves like the previous dict with keys including 'Ep', 'index', and 'current'. Use result.primary for the base-unit Ep scalar, result.table for a tidy Metric/Value display table, and result.show() for pretty or plain printing.

Peak-potential diagnostics accept plot-view options such as 'derivative', so cv.peak_potential({'derivative': 1, ...}) can overlay the selected peak on a derivative trace while keeping the peak-picking analysis on the selected CV data.

Peak selection defaults to 'peak kind': 'both' because cathodic/anodic current sign conventions vary. Set 'peak kind': 'infer' to choose maxima when the selected current rises toward the feature and minima when it falls, or set 'max'/'min' explicitly. With a guess potential, automatic peak prominence is estimated from a local window around the guess when possible and falls back to the global noise estimate; an explicit 'peak prominence' overrides this. Set 'noise window': None to disable peak-finding smoothing.

Example

```
peak = cv.peak_potential({'segment': 1, 'guess potential': −2.05})
Ep = peak['Ep']
idx = peak['index']
peak.table
```

cv.peak_current(options=None)

Description: Calculates tangent-corrected peak current for the selected CV peak.

Parameters

- options (dict or PeakCurrentOptions): peak selection plus tangent baseline controls.
- Peak-current fallback: if peak_potential() cannot locate a local extremum, peak_current() defaults to 'peak fallback': 'highest current', which uses the largest absolute current in the selected segment and records 'peak source' as 'highest

current fallback'. Use 'peak fallback': 'guess potential' to treat 'guess potential' as an exact potential, or 'peak fallback': None to keep strict peak-finding errors.

Key options

Option	Default	Type	Choices	Description
segment				CV segment to analyze.
segments				One or more CV segments to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
tangent potential		float or None		Potential at which to anchor tangent baseline fitting.
tangent range	auto	str or float or list[float] or tuple[float, float]		Potential range for tangent baseline fitting.
tangent min points		int or None		Minimum number of points used for tangent baseline fitting.
percent threshold		float or None		Percent threshold used in peak or tangent selection.
plot				Whether to plot the result.
print				Whether to print a result summary.
troubleshoot				Print additional troubleshooting output.

When overlaying peak-current diagnostics on an existing cv.plot() or multiplot() axes, set {'plot': True, 'plot CV': False, 'new plot': False}. This draws the peak marker, tangent baseline, and current-distance line without redrawing the underlying CV trace.

Returns: CVAnalysisResult, an AnalysisResult child that behaves like the previous dict with keys including 'ip', 'Ep', tangent-line diagnostics, and fit indices. Use result.primary for the base-unit ip scalar; result.table scales values to readable display units such as microamps.

Example

```
peak = cv.peak_current({'segment': 1, 'guess potential': −1.8, 'tangent range': 'auto'})
ip = peak['ip']
tanline = peak['tangent line']
peak.show()
```

cv.peak_width(options=None)

Description: Measures tangent-corrected full peak width for the selected CV peak.

Parameters

- options (dict or PeakWidthOptions): peak selection, tangent baseline, fractional level, plotting, and printing controls.

Key options

The default 'level' is 0.5, so peak_width() reports full width at half peak current. The level must be greater than 0 and less than 1. Crossings are linearly interpolated internally and reported in scan order as 'E leading' and 'E trailing'; there is no public side, mode, or interpolate option.

Returns: CVAnalysisResult with keys including 'width', 'E leading', 'E trailing', 'Ep', 'ip', 'level', 'level current', tangent-line diagnostics, and fit indices. result.primary is the base-unit width in volts, and result.table shows the unit-scaled Metric/Value display.

Example

```
width = cv.peak_width({'segment': 1, 'guess potential': −1.8, 'level': 0.5, 'tangent range': 'auto'})
full_width = width['width']
E_leading = width['E leading']
E_trailing = width['E trailing']
width.show()
```

cv.half_peak_potential(), cv.half_wave_potential(), cv.wave_info()

Defaults are exploratory. Single-feature helpers such as peak_potential(), peak_current(), peak_width(), half_peak_potential(), and peak_info() choose the largest detected feature when segment and guess potential are omitted. Paired-wave helpers such as half_wave_potential() and wave_info() default to segment 1 paired with segment 2 when segments is omitted. For final analysis, specify segment, segments, guess potential, or exact potential so the intended feature is recorded in the workflow.

Complex CV analyses accept per-CV potential lists through plural aliases. describe_options shows these as one row with (s), such as 'guess potential(s)', 'tangent potential(s)', 'non-catalytic cv(s)', and 'scan rate(s)'. Use 'guess potentials', 'exact potentials', 'tangent potentials', 'peak potentials', 'non-catalytic guess potentials', or FOWA 'redox potentials' when each CV needs its own value. A scalar 'guess potential' keeps the existing running-guess behavior in multi-segment analyses. For paired-peak analyses such as trumpet_analysis and nicholson_analysis, a flat two-value 'guess potential' is treated as a shared paired guess when there are more than two CVs; with exactly two CVs, use nested pairs like [[g1a, g1b], [g2a, g2b]] when each CV needs a paired guess. If both singular and plural spellings are supplied for the same option, eCAT raises an option error.

Description: Convenience methods for CV peak and wave diagnostics. These methods use the same segment, smoothing, plotting, and peak-current option families as peak_potential(), peak_current(), and peak_width().

Parameters

- options: CV selection and diagnostic display controls.

Key options

Option	Default	Type	Choices	Description
segment				CV segment to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
peak prominence				Minimum peak prominence for automatic peak detection.
tangent range				Potential range for tangent baseline fitting.
plot				Whether to plot the result.
print				Whether to print a result summary.
troubleshoot				Print additional troubleshooting output.

Returns: CVAnalysisResult objects, AnalysisResult children that preserve dictionary access. peak_info() includes tangent-corrected W1/2 plus a width status. wave_info() includes W1/2 and width status for both peaks plus |ip,return/ip,forward|

from tangent-corrected peak currents. Width failures are reported as Unavailable with the calculation reason retained in result.diagnostics.

Example

```
info = cv.wave_info({'segment': 1, 'guess potential': −2.0, 'plot': True})
```

Complex Data Analysis

fowa(cvs, options=None)

Description: Performs foot-of-the-wave analysis for one CV or a list of CVs. Printed output separates shared settings from per-CV table columns and shows fit quality diagnostics.

Parameters

- cvs: catalytic cv object or list of cv objects.
- options (dict or FOWAOptions): reference-current, redox-potential, fitting, mechanism, and plotting controls.

Key options

Option	Default	Type	Choices	Description
non catalytic current				Manual non-catalytic reference current.
non catalytic cv				Single non-catalytic reference CV.
ip0				Non-catalytic reference peak current.
redox mode	half wave	str	half wave, half peak, manual	Method used to resolve the reference redox potential.
redox potential				Manual redox reference potential.
fit basis	x	str	x, y	Axis or quantity used to select the fit region.
fit range	[0.0, 0.2]	list[float] or tuple[float, float]		Range of the transformed or raw axis included in fitting.
wave range		list[float] or tuple[float, float] or None		Manual FOWA catalytic-wave potential range; may be one range or one range per CV.
background correction	tangent	str or None	tangent, start current	Background correction method used before kinetic analysis.
diagnostic y axis	i/ip0	str	i/ip0, current	Y axis used for FOWA diagnostic multiplot output.
plot				Whether to plot the result.
plot all				Whether to show diagnostic or child plots.
plot fit				Whether to overlay fitted curves or lines.
legend mode			auto, colorbar, discrete	Legend mode for discrete or gradient color encodings.
colorbar height				Scale factor for colorbar height in gradient legends.

Option	Default	Type	Choices	Description
scale				
mechanism	EC'	str		Electrocatalytic mechanism label or model choice.
catalyst electrons	1	float		Number of catalyst redox-wave electron-transfer processes used in kinetic equations.
turnover electrons	1	float		Catalyst equivalents or electrons used per turnover in kinetic equations.
min r2	0.98	float		Minimum recommended R2 threshold.
min fit points	20	int		Minimum recommended number of fit points.

Manual redox mode: when 'redox mode' is 'manual', FOWA still reports Catalytic Ecat/2 and Ecat/2 - E1/2 using the manual redox potential as E1/2. Set 'ecat shift warning threshold' to False to suppress that status check without removing other fit-quality status values.

FOWA diagnostic segment display: by default, plot all diagnostic multiplots keep the full CV traces. Set 'plot segments' to an integer or list of integers to restrict only the diagnostic plotted traces; the FOWA analysis segment is still controlled separately by 'segment' or 'segments'.

FOWA plot side effects: plot=True creates fit/diagnostic Matplotlib figures, and plot all=True also creates diagnostic multiplots using diagnostic y axis, default i/ip0. Use diagnostic y axis="current" for raw-current diagnostics. These figures are side effects; the returned AnalysisResult stores the summary table in result.table.

- Default redox mode is 'half wave'. If you provide 'redox potential', also set 'redox mode': 'manual'; eCAT raises an option error instead of silently switching modes.

Returns: AnalysisResult summary table. Use result.table for the FOWA table; use result.table.columns, result.table.loc, result.table.attrs, or result.table['kobs'] for DataFrame operations. FOWA tables include a compact Status column; detailed warning text is kept in result.diagnostics['full results'] and result.table.attrs['warnings'].

FOWA printed summaries: with print=True, FOWA prints a vertical Field/Value summary table for shared settings and reference information, then the symbolic kobs equation with definitions, before the main FOWA result table. It does not print a second equation with n values substituted. With pretty print=False, the same summary shape is printed as plain text.

- Set 'warnings': False to suppress Python warnings while preserving Status and hidden warning-detail metadata in the returned table.
- FOWA unit metadata includes entries such as ip0 = A, redox potentials = V, R2 = dimensionless, and kobs/TOFmax = s^-1.

FOWA and Tafel return AnalysisResult objects. Use result.table for the primary table; FOWA does not pretend to be a DataFrame, so use result.table.columns, result.table.loc, result.table.attrs, and result.table["kobs"]. Plot side effects are not encoded in the return value for FOWA, Sevcik, or Tafel workflows; use Matplotlib state (for example plt.gcf() or returned Axes from lower-level plotting calls) when you need to further customize figures.

Scatter-fit plot colors: fit_model, fit_rate, fit_peak_current, fit_peak_potential, sevcik_analysis, trumpet_analysis, and multi_scatterplot use matching point and fit-line colors by default. Pass 'fit color' to override the fit line explicitly. For functions that draw multiple fits, 'fit color' may be a list; 'fit colors' is accepted as the plural alias, and describe_options displays this as 'fit color(s)'. Colors are consumed in plotted fit order, and the last color repeats if the list is shorter than the number of fits. If both spellings are supplied, eCAT raises an option error.

Example

```
table = e.fowa(cvs, {
    'non-catalytic cv': blank_cv,
```

```
    'redox mode': 'half wave',
    'fit basis': 'y',
    'fit range': [1, 5],
    'plot all': True,
})
```

- Peak-current fits support 'y mode' and 'y0' to fit raw, delta, negative delta, ratio, or fractional current responses before any y transform.

fit_peak_current(cvs, options=None)

Description: Fits peak current against scan rate or another resolved x quantity. Normalized CVs with an i/ip0 axis are supported. The fit uses the shared fit_model engine; use 'fit model' for linear, power, power offset, exponential, Michaelis-Menten, logistic, callable, or formula fits.

Parameters

- cvs: list of CVs.
- options (dict or FitIpOptions): peak-current extraction, fit, plotting, and print controls.

Key options

Option	Default	Type	Choices	Description
segment				CV segment to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
tangent range	auto	str or float or list[float] or tuple[float, float]		Potential range for tangent baseline fitting.
fit				Whether to fit the analysis result.
fit indices				Indices, mask, or index windows included in a fit.
x transform				Transform applied to x values.
plot				Whether to plot the result.
plot all				Whether to show diagnostic or child plots.
print				Whether to print a result summary.
legend				Whether to show a plot legend.
metric				Metric column or quantity to analyze.

Returns: ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits in new notebooks; tuple unpacking is not part of the beta API.

Example

```
result = e.fit_peak_current(cvs, {'segment': 1, 'guess potential': −1.5, 'plot all': True})
```

- Y adjustment options: 'y mode' may be raw, delta, negative delta, ratio, or fractional; 'y0' may be a scalar or mapping. The adjustment is applied before any y transform and before fitting. Plotted y-axis labels show the y-mode expression

first and then wrap it with any y transform, matching the order used for fitting. Baseline labels use a superscript 0, for example kobs^0. When the input current axis is normalized i/ip0, fit_peak_current labels the fitted peak-current axis as ip/ip0.

- For plot-all diagnostics, use 'plot segment' or 'plot segments' to limit the displayed CV traces without changing the analysis 'segment'/'segments'.
- fit_peak_current can infer a varying mole-fraction token such as 0.8xD2O even when the same species also appears at a fixed molar concentration, such as 2.8MD2O.

fit_peak_potential(cvs, options=None)

Description: Fits peak potential versus log10(scan rate) or log10(concentration). Non-positive x values are retained in the returned table but excluded from log-transformed plotting and fitting. The fit uses the shared fit_model engine; use 'fit model' for direct model fits on the resolved x/y points.

Parameters

- cvs: list of CVs.
- options (dict or EpFitOptions): peak-potential extraction and fit controls.

Key options

Option	Default	Type	Choices	Description
segment				CV segment to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
fit				Whether to fit the analysis result.
fit indices				Indices, mask, or index windows included in a fit.
x transform				Transform applied to x values.
plot				Whether to plot the result.
plot all				Whether to show diagnostic or child plots.
print				Whether to print a result summary.
legend				Whether to show a plot legend.

Returns: ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits in new notebooks; tuple unpacking is not part of the beta API.

Example

```
result = e.fit_peak_potential(cvs, {'segment': 1, 'guess potential': −0.4})
```

- Rate fits support 'y mode' and 'y0' to fit raw, delta, negative delta, ratio, or fractional metric responses before any y transform. Plotted y-axis labels show the y-mode expression first and then wrap it with any y transform, matching the order used for fitting. Baseline labels use a superscript 0, for example kobs^0.

fit_model(x_or_result, y=None, model=None, options=None)

Description: Fits a direct scatter model to x/y arrays, a pandas DataFrame, or an AnalysisResult/ScatterFitResult. Supported models are linear, power, power offset, exponential, Michaelis-Menten, and logistic, plus callables and restricted formula strings. Standalone fit_model accepts either the canonical 'fit ...' options or bare aliases such as model, init, bounds, residual, range, ranges, and indices because the function context is already fitting. In fit_rate, fit_peak_current, and fit_peak_potential, use the canonical option names: 'fit model', 'fit init', 'fit bounds', 'fit residual', 'fit max evals', 'fit range', 'fit ranges', and 'fit indices'. With print=True, pretty output is adaptive: simple fits show one Field/Value table with model settings, fit statistics, and fitted parameter values with standard errors inline when available; constrained or complex fits show Fit Model Details and Fit Model Parameters tables with initial values, bounds, final values, and standard errors. Auto uses details for explicit fit init, explicit fit bounds, custom formula/callable models, constrained models such as power offset/logistic/Michaelis-Menten, three or more parameters, or parameters at a bound. Use print fit='summary' or print fit='details' to force the style. Printed equations are general model equations so parameter meanings remain visible; plot labels use fitted numeric equations formatted by sig figs. fit_rate uses the same human table; transforms such as transform mode='log-log' only change the coordinates being fit and do not change or relabel the selected fit model. Use fit model='power' when you want a direct power-law model. Set pretty print=False for a compact dictionary-style summary. Formula strings such as k0 + k1*x + k2*x^2 infer parameter names when possible and support ^ as a power shorthand. Use fit range for one x-value window on the resolved/transformed x axis, fit ranges for multiple named or generated x-value-window fits, and fit indices for row/position windows. Selected points determine the parameters while predictions/residuals are retained for the original input rows.

Example: result = e.fit_model(rate_result, model='michaelis-menten', options={'plot': True, 'print': True})

fit_rate(df, options=None)

Description: Fits a FOWA-derived metric such as kobs against a concentration, scan-rate, or transformed x column. When input DataFrames provide unit metadata, kobs plots include units such as s^-1 in the y-axis label.

Parameters

- df: FOWA AnalysisResult, FOWA output table, or compatible DataFrame.
- Use 'fit indices' for row/position-based selection; [start, stop] uses a Python-style exclusive stop, so [0, 3] selects rows 0, 1, and 2, and None leaves either side open-ended. Use 'fit range' for one x-value window on the resolved/transformed x axis. Use 'fit ranges' for multiple named or generated x-value-window fits; nested windows allow one fit to use disconnected x regions. Use 'fit model' to fit the selected points with a direct model instead of a transformed straight line; 'fit init', 'fit bounds', 'fit residual', and 'fit max evals' control the direct fit.
- fit_rate can infer mole-fraction x values from FOWA-style Compounds or Plot Label columns and labels them as χ(species).
- Result tables produced by FOWA and rate-style analyses use table.attrs['units'] for derived column units where available.
- options (dict or FitRateOptions): metric, x selection, filtering, fit, and plotting controls.

Key options

Option	Default	Type	Choices	Description
metric	kobs	str or None		Metric column or quantity to analyze.
x column	auto	str or int or None		DataFrame column used as x data.
species		str or None		Chemical species used to resolve concentration or metadata.
x transform		str or float or int or None		Transform applied to x values.
plot log log	False	bool		Whether to include a log-log plot.

Option	Default	Type	Choices	Description
exclude warnings	False	bool		Exclude rows or fits that emitted analysis warnings.
exclude low r2	False	bool		Exclude fits whose R2 is below the requested threshold.
fit indices		object or None		Indices, mask, or index windows included in a fit.
fit	True	bool		Whether to fit the analysis result.
print	True	bool		Whether to print a result summary.
plot	True	bool		Whether to plot the result.
legend	False	bool or str		Whether to show a plot legend.
local slope mode	adjacent	str	adjacent, gradient	Method for calculating local slopes.
print local slopes	False	bool		Whether to print local slope diagnostics.
fit ranges		object or None		Named or unnamed fit windows for multiple fit_rate fits. Use a dict for named fits or a list for generated labels.

Returns: ScatterFitResult, an AnalysisResult child, with table, fits, fit_table, figure, axes, and summary metadata. Use result.table and result.fits; tuple unpacking is not part of the beta API.

Example

```
fowa_result = e.fowa(cvs, {'print': False, 'plot': False})
rate = e.fit_rate(fowa_result, {
    'metric': 'kobs',
    'transform mode': 'log-log',
    'fit ranges': {'early': [−11, −4], 'full tail': [−11, None]},
})
```

- For example, {'y mode': 'fractional'} fits y/y0 - 1, where y0 defaults to the first valid raw y value in each series.

sevcik_analysis(cvs, options=None)

Description: Runs the bulk diffusion-controlled Sevcik peak-current analysis over CVs. Scan-rate series always fit peak current against the square root of scan rate; use surface_coverage_analysis() for surface-confined peaks that are linear in scan rate.

Parameters

- cvs: list of CVs.
- options (dict or SevcikOptions): peak-current, scan-rate, fit, and plot controls.

Key options

Option	Default	Type	Choices	Description
segment		int or None		CV segment to analyze.
guess potential		float or None		Initial potential guess for peak or wave selection.
tangent range	auto	str or float or list[float] or tuple[float, float]		Potential range for tangent baseline fitting.

Option	Default	Type	Choices	Description
fit				Whether to fit the analysis result.
fit indices		object or None		Indices, mask, or index windows included in a fit.
x transform				Transform applied to x values.
plot	True	bool		Whether to plot the result.
plot all	False	bool		Whether to show diagnostic or child plots.
print	True	bool		Whether to print a result summary.
legend	False	bool or str		Whether to show a plot legend.

Returns: ScatterFitResult, an AnalysisResult child. When plot=True, Sevcik-style helpers create/update Matplotlib figures as a side effect; the fit result remains the returned analysis object.

Example

```
result = e.sevcik_analysis(cvs, {'segment': 1, 'guess potential': −1.2, 'plot': True})
```

reversibility_analysis(cvs, options=None)

Description: Assesses electron-transfer reversibility and chemical return-current behavior for one chemical condition measured at three or more distinct scan rates. Replicates remain in the result table while trend fits use scan-rate means. Five or more rates are recommended.

Parameters

- cvs: one CV scan-rate series; group mixed conditions first with e.group(...).
- options: phase, segment/peak controls, n, D, C/species, area, temperature, agreement tolerance, print, and plot controls.

Decision tree

- Bulk: convert eligible n Delta Ep to Nicholson psi and Lambda; Matsuda-Ayabe uses Lambda >= 15 for reversible and 10^[−2(1+alpha)] < Lambda < 15 for quasi-reversible. Nicholson k0 is used only for 0.1 <= psi <= 7.
- If D is omitted, exact species concentration plus area and temperature allow branch-specific Sevcik Dapp estimates. Both branches must meet min R2 and agreement tolerance.
- A series that remains reversible reports a k0 lower bound. Irreversible behavior is only reported when large peak separation is corroborated by both Ep-Ep/2 and Ep-v asymptotes.
- Surface: require ip linear in scan rate and compare Delta Ep with max(configured tolerance, three median potential increments). Laviron is eligible only for at least two n Delta Ep > 200 mV points.
- Chemical conclusion: return/forward peak-current ratios within agreement tolerance of unity are chemically reversible over the observed timescale; persistent deviations or recovery toward unity at faster scans indicate coupled chemistry. Sparse or conflicting evidence is indeterminate.

Returns: AnalysisResult with per-CV table, scan-rate means, exact decision evidence, kinetic estimates or bounds, warnings, and diagnostic figures.

Example

```
result = e.reversibility_analysis(cvs, {
  'phase': 'bulk',
  'guess potential': −1.0,
  'num electrons': 1,
```

```
    'D': 1e-5,
})
```

surface_coverage_analysis(cvs, options=None)

Description: Estimates surface coverage Gamma (mol/cm^2) and total electroactive loading (mol) from the independent ip-versus-scan-rate and tangent-corrected charge equations. Loading remains available without area; Gamma requires area.

Equations

- ip = n^2 F^2 A Gamma v / (4 RT)
- Q = n F A Gamma; loading = A Gamma

Automatic integration searches for tangent-baseline returns around each peak. Use integration range for an explicit potential window. Branch and method disagreements beyond agreement tolerance warn and retain both estimates rather than silently averaging them.

Example

```
coverage = e.surface_coverage_analysis(cvs, {
    'segments': [1, 2],
    'guess potential': [−0.1, −0.1],
    'integration range': 'auto',
})
```

plateau_current(cvs, options=None) / cv.plateau_current(options=None)

Description: Analyzes scan-rate-independent catalytic plateau currents. Use e.plateau_current(cvs, options) for grouped scan-rate validation, or cv.plateau_current(options) as a single-CV convenience wrapper around the same workflow. When non-catalytic CVs are supplied with plot all=True, eCAT shows the reference/Sevcik-style diagnostic first, then catalytic plateau extraction, then plateau validation.

Parameters

- cvs: catalytic CV or list of catalytic CVs.
- options (dict or PlateauCurrentOptions): peak-current extraction, non-catalytic reference, plateau-validation, formula, print, and plotting controls.
- Peak-current extraction: plateau_current() now delegates no-peak fallback behavior to peak_current(). Use 'peak fallback' to choose 'highest current' (default), 'guess potential', or None for strict errors. For non-catalytic reference series, plateau_current() may still reuse a successful extraction potential from a nearby scan rate as an exact-potential hint.

Key options

non-catalytic cv / non-catalytic cvs, ip0, ip0 scan rate, ip0 sqrt scan rate slope, formula mode, validate plateau, require plateau, plateau average method, plot all

Returns: DataFrame summary with plateau current, reference-current source, formula mode, and kobs.

Example

```
result = e.plateau_current(cvs, {
    'guess potential': −1.6,
    'non-catalytic cvs': blanks,
    'turnover electrons': 2,
    'plot all': True,
})
```

trumpet_analysis(cvs, options=None)

Description: Analyzes scan-rate-dependent peak splitting from paired CV segments and returns a ScatterFitResult, an AnalysisResult child. Peak selection and fit controls are routed through typed options.

Parameters

- cvs: list of CVs.
- options (dict or TrumpetOptions): peak selection, scan-rate transform, fit, and plot controls.

Key options

Option	Default	Type	Choices	Description
segment	1	int or None		CV segment to analyze.
guess potential		float or None		Initial potential guess for peak or wave selection.
exact potential		float or None		Exact potential to use for peak or current extraction.
fit				Whether to fit the analysis result.
fit indices		object or None		Indices, mask, or index windows included in a fit.
x transform				Transform applied to x values.
plot	True	bool		Whether to plot the result.
plot all	False	bool		Whether to show diagnostic or child plots.
print	True	bool		Whether to print a result summary.
legend	False	bool or str		Whether to show a plot legend.

Returns: ScatterFitResult, an AnalysisResult child. When plot=True, Sevcik-style helpers create/update Matplotlib figures as a side effect; the fit result remains the returned analysis object.

Example

```
result = e.trumpet_analysis(cvs, {'segments': [1, 2], 'guess potential': −0.5})
```

nicholson_analysis(cvs, options=None)

Description: Estimates heterogeneous electron-transfer kinetics from paired peak separations in a scan-rate series. Nicholson analysis requires a diffusion coefficient and is most appropriate when peak separation reflects electron-transfer kinetics rather than referencing, uncompensated resistance, adsorption, or background subtraction artifacts.

Parameters

- cvs: CV or list of CVs containing the redox couple to analyze.
- options (dict or NicholsonOptions): segment selection, D, psi source, valid delta-Ep range, fit, print, and plotting controls. See e.describe_options("nicholson_analysis").

Key options

D, segment, num electrons, psi source, fit through origin, exclude invalid delta ep, plot all, plot diagnostic, warn ir drop

Returns: AnalysisResult with result["data"] and result.table for the Nicholson point table and result["summary"] / result.summary for equation metadata, fit statistics, and kinetic values such as k0. With print=True, eCAT prints the Nicholson equation, a summary table, and the input/result table. With plot all=True, eCAT makes one CV half-wave diagnostic plot and one Nicholson fit plot.

Example

```
result = e.nicholson_analysis(cvs, {
  'segment': 1,
  'guess potential': −0.5,
  'D': 1e-5,
```

```
    'plot all': True,
})
result['summary']
```

scale_current(cvs, options=None)

Description: Returns copied CVs whose raw current columns are multiplied by a manual scale factor or by a factor resolved from reference-wave peak currents. reference mode="single" uses one selected peak; reference mode="both" uses explicit segments when provided, otherwise segment and segment+1, commonly ip,c and ip,a, to compute one signed scale factor. Original CV objects are not modified.

Parameters

- cvs: list of CVs.
- options (dict or ScaleCurrentOptions): scale source, reference-CV, peak-current controls such as guess potential, print, and plot controls. See e.describe_options("scale_current").

Key options

Option	Default	Type	Choices	Description
reference index	0	int		Index of the reference CV used for current normalization or scaling.
scale		float or list[float] or None		Multiplier applied to raw current columns by scale_current.
segment				CV segment to analyze.
segments				One or more CV segments to analyze.
guess potential				Initial potential guess for peak or wave selection.
exact potential				Exact potential to use for peak or current extraction.
tangent range				Potential range for tangent baseline fitting.
tangent potential				Potential at which to anchor tangent baseline fitting.
print				Whether to print a result summary.
plot all	False	bool		Whether to show diagnostic or child plots.
reference mode	single	str	single, both	Reference mode for import reference shifting or scale_current reference-wave selection.

Returns: cv or list[cv], matching input shape, with scaled raw current data.

Example

```
scaled = e.scale_current(cvs, {'reference index': 0, 'reference mode': 'both', 'segment': 1})
```

normalize_current(cvs, options=None)

Description: Returns copied CVs with an i/ip0 column for FOWA-style normalized current plotting and analysis. Original CV objects are not modified.

Parameters

- cvs: cv or list[cv].

- options (dict or NormalizationOptions): ip0 source, reference peak-current controls, and optional plot/print controls.

Key options

Option	Default	Type	Choices	Description
ip0		float or list[float] or None		Non-catalytic reference peak current.
non catalytic current				Manual non-catalytic reference current.
non catalytic cv				Single non-catalytic reference CV.
non catalytic cvs				List of non-catalytic reference CVs.
segment				CV segment to analyze.
guess potential				Initial potential guess for peak or wave selection.
tangent range				Potential range for tangent baseline fitting.
plot all				Whether to show diagnostic or child plots.
print				Whether to print a result summary.
pretty print				Use rich table display when printing object lists or summaries.

Returns: cv or list[cv], matching input shape.

Example

```
norm_cvs = e.normalize_current(cvs, {'reference cv': blank_cv, 'segment': 1, 'plot all': True})
```

normalize(cvs, options=None)

Description: Returns copied CVs with physical dimensionless potential and/or current axes for homogeneous or heterogeneous normalization. Original CV objects are not modified.

Parameters

- cvs: cv or list[cv].
- options (dict or NormalizeOptions): physical constants and normalization mode. See e.describe_options("normalize").

Key options

- E0 creates a stored column named "Dimensionless Potential".
- Current normalization creates a stored column named "Dimensionless Current" when D, concentration, electrode area, and scan rate are available; electrode area and scan rate may be collected automatically from the CV object.
- C and C unit may be provided explicitly. If they are omitted, species may be used to exact-match cv.compounds and pull the corresponding cv.concentrations entry; concentration is not guessed without species.
- Temperature defaults to cv.temperature whenever normalize creates a dimensionless axis. For current-axis normalization, electrode area defaults to cv.electrode_area and scan rate defaults to cv.scan_rate when not provided explicitly.
- D is expected in cm^2/s; concentration is converted internally to mol/cm^3; electrode area is cm^2; scan rate is V/s.

Option	Default	Type	Choices	Description
normalize				Whether to normalize current or axes during processing.
normalize params				Parameters used for legacy normalization.
plot				Whether to plot the result.
print	False	bool		Whether to print a result summary.

Behavior notes

- The wrapper copies inputs; cv.normalize(...) mutates a single CV and returns self.
- Normalized CVs use the normalized axes by default for x(), y(), and plot() when those axes were created. Explicit x axis or y axis options can still select raw columns such as Potential or Current.
- For a series, normalize(..., {'print': True}) prints the symbolic normalization equation once, reports the shared mode outside the table, then shows a parameter table with parameters as rows and group-index columns. For one CV, the value column is labeled Value. Use 'pretty print': False for plain-text output.
- Default plot labels use compact symbols: θ for dimensionless potential, Φ for homogeneous dimensionless current, and χ for heterogeneous dimensionless current. The full equations are shown when normalize(..., {'print': True}) is used.
- normalize_current(...) is separate from physical dimensionless normalization. It creates i/ip0 from the raw Current column; it does not compute Phi/Phi_p0.

Returns: cv or list[cv], matching input shape.

Example

```
norm = e.normalize(cvs, {'mode': 'homogeneous', 'E0': 0.0, 'D': 1e-5, 'C': 10, 'C unit': 'mM'})

norm = e.normalize(cv, {'mode': 'homogeneous', 'E0': −1.0, 'D': 1e-5, 'species': '[Co]'})
```

tafel_analysis(cv_or_list, TOF_max, thermodynamic_potential, redox_potential, options=None)

Description: Calculates and plots Tafel-style turnover-frequency curves for one catalytic CV or a CV series. Multi-CV calls use the same shared label, color, gradient, legend, and colorbar behavior as multiplot.

Parameters

- cv_or_list: catalytic CV, or a list of catalytic CVs.
- TOF_max: scalar value, or one value per CV. thermodynamic_potential and redox_potential set the overpotential relationship.
- options (dict or TafelOptions): Tafel controls plus common multiplot styling options such as labels, gradient by, legend mode, and colorbar tick labels.

Key options

Option	Default	Type	Choices	Description
overpotential range	[0, 1]	list[float] or tuple[float, float]		Potential range used for Tafel or overpotential analysis.
color	black	str or None		Primary plot color.

Returns: AnalysisResult with result["data"] / result.table for Tafel points, result["summary"] for the summary table, and result.axes for the plot axes.

Example

```
taf = e.tafel_analysis(cvs, TOF_values, E_thermo, E_redox, {'gradient by': 'concentration', 'legend mode': 'colorbar'})
```

Simulation namespace

Overview

Simulation helpers live under e.simulation. They are supported advanced workflows for CV mechanism simulation and fitting, with explicit mechanism, species, diffusion, cell, and fitting assumptions. ElectroKitty is optional at import time, but simulate_cv(), fit_cv(), and fit_cvs() need the simulation extra installed when they call the backend.

Install the backend-enabled extra with:

```
python -m pip install "ecat[simulation]"
```

Use e.describe_options("simulation.cv_data"), e.describe_options("simulation.simulate_cv"), and e.describe_options("simulation.fit_cv") for the current option tables. Short names such as describe_options("fit_cv") are accepted when unambiguous.

Simulation object model

SimulatedCVInput is the potential/time/current input object consumed by simulation and fitting. cv_program() creates a program-only input; cv_data() extracts a measured CV region and keeps measured current for fitting. Incubation time and quiet time are input metadata. SimulatedCV subclasses cv for plotting and CV-analysis compatibility, but SimulatedCV.data contains simulated current only. Measured-vs-simulated overlays belong to SimulationFitResult.plot() and SimulationGroupFitResult.plot().

show(), plot(), and with_* helpers are designed for notebook workflows: show() prints setup, parameter, and concentration-state tables; plot() visualizes inputs or results; and with_scan_rate(), with_incubation_time(), with_param(), with_params(), with_input(), and with_mechanism() return changed copies without mutating the original object. SimulatedCV.input_params retains normalized entered parameters so reruns and fitting never reapply pre-equilibrium or incubation to an already prepared state.

Simulation parameter model

Use concentrations plus diffusion for species setup. The species mapping is input sugar only when written as per-species type/C/D entries; prepared/output params normalize it into concentrations and diffusion.

Simulation units

Simulation parameters use SI-derived public units. Potentials are V; time and incubation_time are s; current is A; scan rate is V/s; bulk concentrations are mol/m^3; surface coverages are mol/m^2; diffusion is m^2/s; electrode area is m^2; uncompensated resistance is ohm; temperature is K; and cell.Cdl is total double-layer capacitance in F. Bulk activities default to a 1000 mol/m^3 standard concentration (1 M); surface activities use a separate 1 mol/m^2 standard coverage.

cell.Cdl="auto" estimates total farads from measured forward/reverse current separation. Backend adapters that require areal capacitance, including ElectroKitty, convert internally with cell.Cdl / cell.A, so do not divide by area before passing cell.Cdl.

Electrochemical k0 is m/s. Chemical k, kf, and kb are reaction-order dependent: first-order rates are s^-1, while higher-order units follow the amount basis of the reaction phase. Equilibrium K is dimensionless by default under the activity convention; unit-bearing bulk strings such as "2 M^-1" are converted using the bulk activity standard and any non-ideal gamma values.

Kinetics and reactions are user-facing mechanism parameter sections. kinetics entries describe electrochemical E steps with E0, k0, and alpha. reactions entries describe chemical C steps and may use irreversible k, reversible kf/kb, or physical equilibrium parameters such as K with k_exchange or koff. Fits can vary physical paths like reactions.0.K; eCAT solves each trial's pre-equilibrium and compiles backend-ready kf/kb rates under _compiled before simulation.

Every species in a selected pre-equilibrium reaction must have an explicit entered concentration, including zero. eCAT derives numerical conservation constraints directly from the selected reaction stoichiometry; separate pool declarations are not used. This solver reports reaction rank, dependent equations, conservation rank, and residuals. It does not infer elemental composition or charge from arbitrary species names, so it does not claim that a user-named mechanism is chemically balanced.

Reaction-Local Equilibria And Chemical Incubation

A reaction-local K is dimensionless by default and uses activities with stoichiometric powers. For species j, a_j = gamma_j x_j / x_standard, where x is bulk concentration or surface coverage. K is the product of product activities divided by the product of reactant activities. Coefficient notation such as 2A = B and repeated notation such as A + A = B are equivalent; both square a_A. Activity coefficients default to 1 and are omitted from printed species tables unless a value differs from 1.

K determines the forward/reverse ratio, not the equilibration speed. Add k_exchange in s^-1 as the phase-standard exchange-frequency scale, or add koff when the reverse rate is known. eCAT compiles these physical inputs to the amount-basis kf/kb values required by the backend for each simulation and fit evaluation. Optional reference_concentration or reference_coverage values must be positive and finite.

Reversible reactions with K participate in the initial pre-equilibrium by default. Set equilibrate=False on a reaction that must remain reversible and dynamic but should not add an algebraic starting-state constraint. This is useful for a dependent cycle edge or for chemistry that begins only during incubation or the CV. Consistent dependent cycles are accepted; inconsistent K constraints raise an error with the numerical residual.

Preparation order is: entered concentrations, stoichiometric pre-equilibrium, bulk homogeneous chemical incubation, backend quiet-time hold, then the CV potential program. eCAT performs the pre-equilibrium and incubation stages before calling ElectroKitty. Incubation integrates bulk chemical C steps only; surface and mixed-phase steps remain backend-only. It excludes electron transfer, diffusion, Ru, Cdl, and applied potential. The default incubation_time is 0 s.

Cross-phase bulk/surface pre-equilibrium remains unsupported because bulk amounts and surface coverages require a volume-to-area conversion and a site-density convention. Such pre-equilibria raise a clear error rather than silently combining incompatible amount bases. Dynamic surface and mixed-phase steps remain available to the backend during the CV.

Example

```
program = e.simulation.cv_program(Ei=0, E_low=-1.6, scan_rate=0.1, incubation_time=30)
params['reactions'] = {
    'A+B=C': {'K': 25.0, 'k_exchange': 10.0},
    'C=D': {'K': 4.0, 'k_exchange': 2.0, 'equilibrate': False},
}
result = e.simulation.simulate_cv(program, mechanism, params)
result.show({'print setup': False, 'print states': True})
```

Section	Purpose	Example paths
concentrations	Initial bulk/surface amounts.	concentrations.bulk.Substrate
diffusion	Mobile-species diffusion coefficients.	diffusion.Substrate
kinetics	Electrode-transfer parameters.	kinetics.0.E0, E0_0
reactions	Chemical-step rates or physical equilibrium parameters.	reactions.0.k, reactions.1.K
cell	Temperature, resistance, capacitance, electrode area.	cell.T, cell.Ru, cell.Cdl, cell.A
spatial	Backend mesh/transport presets and settings.	spatial=fast, spatial.nx

activity	Bulk concentration/surface coverage standards and optional activity coefficients.	activity.standard_concentration, activity.standard_coverage, activity.gamma.A

simulation.compile_mechanism(mechanism, params=None)

Description: Parses a mechanism preset, custom eCAT mechanism string, or MechanismSpec into the normalized mechanism object used by simulation and fitting.

Custom eCAT mechanisms accept conventional positive-integer coefficients and ElectroKitty-style repeated species as equivalent stoichiometry. For example, C:A+2B=C and C:A+B+B=C are equivalent. eCAT preserves the entered equation for display and reaction paths, then privately expands coefficients before calling ElectroKitty and uses the installed backend parser species order for positional parameter arrays. A leading integer is always a coefficient, so literal species names beginning with digits are unsupported.

```
mechanism = e.simulation.compile_mechanism("C:A+2B=C", params)
# Equivalent ElectroKitty-style input: C:A+B+B=C
```

Parameters

- mechanism: preset name such as E, EE, EC, ECE, EC'/Ecat, Square, or a custom eCAT mechanism string.
- params: optional parameter dictionary used when a mechanism preset needs species or surface context.

Accepted presets

Preset	Meaning	Notes
E	One electron-transfer step.	Useful for a single reversible redox couple.
EE / E,E	Two electron-transfer steps.	Useful for sequential redox waves.
EC	Electron transfer followed by a chemical step.	Basic coupled chemistry model.
ECE	Electron transfer, chemical step, electron transfer.	Common redox-mediated mechanism pattern.
EC' / Ecat	Catalytic EC-prime mechanism.	Chemical regeneration/catalysis shorthand.
Square	Four-state square scheme.	Compiles to two E steps and two C steps.
Square*	Surface-confined square shorthand.	Uses surface-style species when appropriate.

Returns: MechanismSpec with parsed steps, electrochemical steps, chemical steps, and species ordering used by backend adapters.

Example

```
mechanism = e.simulation.compile_mechanism("EE", params)
```

simulation.cv_program(...)

Description: Builds a SimulatedCVInput potential/time waveform for a simulated CV without measured current.

Parameters

- Ei, E_low, E_high, Ef: initial, lower, upper, and final potentials.
- direction, segments, points_per_segment, scan_rate: waveform direction, number of CV segments, point density, and scan rate in V/s.
- quiet_time: optional backend hold at the initial potential, stored as metadata and materialized only when preparing backend input.
- incubation_time: bulk homogeneous chemical-only pre-run duration in s, stored as input metadata; defaults to 0.

Behavior notes

- cv_program() stays quiet by default; use program.show() for setup and program.plot() to inspect the potential program.
- program.plot({"plot quiet time": True}) draws quiet time at negative time. Stored E/t arrays are not padded with quiet-time points.
- program.with_scan_rate(rate) returns a new input with the same potential path and a new timebase; program.with_incubation_time(seconds) returns a copy with changed pre-run chemical time.

Returns: SimulatedCVInput without measured current. It can be simulated directly, but cannot be fitted until measured current is added through cv_data().

Example

```
program = e.simulation.cv_program(Ei=0.0, E_low=-1.6, E_high=0.0, Ef=0.0, scan_rate=0.1, incubation_time=0)
program.plot({'plot quiet time': True})
```

simulation.cv_data(cv, options=None)

Description: Converts an observed CV or CV-like object into SimulatedCVInput with measured current, potential, time, scan-rate metadata, source metadata, optional trimming, optional background correction, and optional Cdl estimation.

Parameters

- cv: real eCAT cv object or CV-shaped object with potential/current data.
- options: selection, trimming, stride, background-correction, Cdl-estimation, and incubation-time controls.

Key options

Option	Default	Type	Choices	Description
potential window	None	list[float] or None		Potential range selected before fitting/simulation input creation.
trim mode	expand	str	expand, pointwise, strict	How potential-window trimming preserves or restricts connected CV segments.
segments	None	int/list/None		CV segment or segments to extract.
stride	auto	int or str		Downsample selected points; auto uses point targets/density.
points / points per volt	None / auto	int/float/str		Target extracted point count or density for fast notebook fits.
background correction	None	str/bool/None	start current, tangent	Subtract measured-current background after selection and before stride.
estimate Cdl	auto	bool or str		Estimate double-layer capacitance from measured current when possible.
incubation time	0	float		Chemical-only pre-run duration stored on the SimulatedCVInput.

Returns: SimulatedCVInput with measured current. fit_cv() and fit_cvs() call this internally when they receive real CV objects directly.

Example

```
fit_input = e.simulation.cv_data(cv, {'potential window': [0, −1.6], 'trim mode': 'expand', 'stride': 10, 'background correction': 'start current'})
fit_input.show()
```

simulation.simulate_cv(input, mechanism, params, options=None, backend="electrokitty")

Description: Runs a CV simulation from a SimulatedCVInput, mechanism, and parameter dictionary, then returns a SimulatedCV result.

Parameters

- input: SimulatedCVInput from cv_program() or cv_data().
- mechanism: preset, mechanism string/list, or compiled MechanismSpec.
- params: simulation parameter dictionary containing concentrations, diffusion, kinetics/reactions, cell, spatial, and optional activity settings.
- backend: simulation backend; ElectroKitty is the supported backend for numerical CV simulation.

Key options

Option	Default	Type	Choices	Description

plot	True	bool		Plot the simulated CV immediately.
plot all	False	bool		Also show backend/debug current columns when present.
current sign	auto	str/int	auto, backend, native, flip, 1, −1	Match or preserve backend current sign.
use quiet time	True	bool		Materialize quiet-time metadata only for backend input.
print setup	False	bool/str	raw	Print simulation setup; raw gives debug-style output.
print params	False	bool/str	compact, raw	Print prepared parameters.
check params	False	bool		Print diagnostic checks for likely parameter issues.
print states	False	bool		Show entered, equilibrated, and incubated amounts for changed species.

Returns: SimulatedCV with simulated current only, prepared params, compiled mechanism, backend result, setup metadata, plot/show helpers, and CV-compatible data access.

Example

```
result = e.simulation.simulate_cv(program, 'EE', params, options={'print params': 'compact'})
result.show({'print setup': True, 'print checks': True, 'print states': True})
```

SimulatedCV and SimulatedCVInput methods

Description: Notebook-facing methods for inspecting, plotting, and rerunning simulation objects without mutating the original object.

Method	Object	Purpose
show(options=None)	input/result/fit result	Display setup, params, checks, stats, corrections, or raw debug output depending on options.
plot(options=None)	SimulatedCVInput	Plot potential program or measured-current input; quiet time can be shown at negative time.
plot(options=None)	SimulatedCV	Plot simulated CV current only.
with_scan_rate(rate)	input/result	Return a new object rerun at a different scan rate.
with_param(path, value)	SimulatedCV	Rerun with one path update, such as reactions.0.k or cell.Cdl.
with_params(params, set=None)	SimulatedCV	Deep-merge parameter updates and optional path updates, then rerun.
with_input(input)	SimulatedCV	Rerun the same mechanism/params on a replacement input.
with_mechanism(mechanism)	SimulatedCV	Rerun with a replacement mechanism.

with_incubation_time(seconds)	input/result	Copy an input or rerun a result with a new incubation time.

Example

```
fast = result.with_scan_rate(0.5)
incubated = result.with_incubation_time(30)
more_substrate = result.with_params({'concentrations': {'bulk': {'Substrate': 2800}}})
faster_rxn = result.with_param('reactions.0.k', 10.0)
```

simulation.fit_cv(input_or_result, mechanism=None, params=None, fit=None, options=None, backend="electrokitty", method="least squares")

Description: Fits one measured CV to one shared mechanism and parameter model. The input can be a real cv object, a fit-ready SimulatedCVInput with measured current, or a SimulatedCV result carrying measured-current context through its input.

Parameters

- input_or_result: real cv, SimulatedCVInput, or SimulatedCV. cv_program() inputs cannot be fitted because they have no measured current.
- mechanism and params: required unless they can be taken from an existing SimulatedCV.
- fit: dictionary controlling varied/fixed parameters, initial values, bounds, and transforms. fit remains the only place that decides fitted versus fixed status.
- method: least squares or an eCAT optimizer strategy; ElectroKitty native fitting remains single-dataset oriented.

Key options

Option	Default	Type	Choices	Description
residual	direct	str	direct, scale, scale linear baseline	Residual model used during optimization.
post correction	None	str/None	scale, vertical shift, scale linear baseline	Final-only reporting/plotting correction.
residual normalization	max_abs_measured	str/None	max_abs_measured	Normalize optimizer residual magnitude.
max nfev	None	int/None		Maximum optimizer evaluations.
progress	notebook	bool/str		Show notebook progress indicator.
print setup / print params	True / True	bool/str		Display setup and fit parameter tables by default.
print stats / print corrections	False / False	bool		Opt-in fit statistics and correction tables.
cv data	None	dict/None		Nested cv_data options used when input is a real CV.

Returns: SimulationFitResult with best_params, simulation_result, measured-current residuals, corrections, summary statistics, plot(), and show().

Example

```
fit = e.simulation.fit_cv(cv, 'EE', params, fit={'vary': ['E0_0', 'E0_1', 'D'], 'bounds': 'auto', 'transform': 'auto'}, options={'cv data': {'potential window': [0, −1.6], 'stride': 10}, 'post correction': 'vertical shift'})
fit.show({'print stats': True, 'print corrections': True})
```

simulation.fit_cvs(inputs_or_results, mechanism=None, params=None, fit=None, per_cv="auto", options=None, backend="electrokitty", method="least squares")

Description: Fits one shared mechanism across multiple CV datasets while allowing selected parameter paths to be dataset-specific.

Parameters

- inputs_or_results: list/tuple of real CVs, fit-ready SimulatedCVInput objects, SimulatedCV results, or a mixed list of those forms.
- fit: unchanged from fit_cv(); it controls which paths are varied or fixed.
- per_cv: "auto", one path string, a list of paths, or False/None. It marks which paths are dataset-specific, not whether they are fitted.
- options: accepts the fit_cv option family plus group conveniences such as cv data and concentration mapping.

Group-fit behavior notes

- per_cv="auto" detects intrinsic dataset differences such as potential program, scan rate, measured current, source label, concentrations, and available cell metadata.
- If a path appears in both fit["vary"] and per_cv, eCAT fits one value per CV. If a per-CV path is fixed, each CV keeps its own fixed value.
- Kinetics, reactions, diffusion, and mechanism are shared by default; include a path in per_cv only when the scientific model needs dataset-specific values.
- Group residual corrections are per-CV and display under Group Fitting Corrections.

Argument/option	Typical value	Purpose
per_cv	auto	Infer dataset-specific paths from the source CVs and metadata.
per_cv	['cell.Cdl', 'concentrations.bulk.Substrate']	Keep selected paths separate by dataset.
concentration mapping	{'PhOH': 'Substrate'}	Map source compound names to simulation species names.
cv data	{'stride': 10, 'estimate Cdl': 'auto'}	Prepare each real CV before fitting.
post correction	vertical shift	Apply final per-CV diagnostic correction.

Returns: SimulationGroupFitResult with shared best_params, best_params_by_cv, simulation_results, measured currents, residuals, corrections_by_cv, summary, plot(), and show().

Example

```
group_fit = e.simulation.fit_cvs(cvs, mechanism, params, fit={'vary': ['reactions.0.kf'], 'bounds': 'auto'}, per_cv=['cell.Cdl', 'concentrations.bulk.Substrate'], options={'cv data': {'stride': 10, 'estimate Cdl': 'auto'}, 'concentration mapping': {'PhOH': 'Substrate'}})
group_fit.show({'print stats': True, 'print corrections': True})
```

Simulation display and reporting

Simulation and fitting objects use .show() for notebook-friendly displays. Setup and final fit parameters print by default for fitting calls; pass print setup=False, print params=False, or progress=False when preparing faster-running notebooks. Use print states=True to compare entered, equilibrated, and incubated species amounts. Use print progress=True only for the detailed Fitting Progression audit table.

Display option	Applies to	Effect
print setup	inputs, results, fits	Show setup; raw mode prints debug dictionaries.
print params	results, fits	Show simulation or fitting parameter tables; compact is available for simulation params.
print checks	SimulatedCV	Show diagnostic parameter checks without rerunning.
print stats	fit results	Show fit cost/residual statistics.
print corrections	fit results	Show final-only correction terms separately from mechanism parameters.
print simulation	fit results	Show final simulated-CV setup.
print states	SimulatedCV	Show changed species across entered, equilibrated, and incubated states.

Defaults and Option Discovery

Defaults precedence: package defaults < loaded TOML defaults < session defaults < per-call options. echem objects and analysis DataFrames expose unit metadata through .attrs['units'] where available; raw echem data still also use object.units for column units.

For describe_options itself, print controls whether anything is emitted, while pretty print only controls output style: {'pretty print': False} prints a plain text table, and {'print': False} suppresses output.

Option names accept spaces, underscores, and hyphens in most notebook-facing APIs, and registered choice values are case-insensitive. For choice-like options, spaces, underscores, and hyphens are also treated interchangeably. If an option explicitly lists 'none' as a valid choice, Python None, 'None', 'off', 'false', 'no', and '0' are interpreted as the canonical string 'none'; omitting the option still uses the default.

describe_options('function_name') reports function-specific option metadata: current defaults from eCAT's defaults system, dataclass-derived types, and function-specific choices and descriptions. This matters for shared option names such as mode, which can mean include/exclude for filter but homogeneous/heterogeneous for normalize.

When an option is resolved automatically, the Description column distinguishes the source. Metadata retrieval is described explicitly, such as using a CV's stored scan_rate, temperature, electrode_area, compounds, or concentrations. Algorithmic auto behavior is described as automatic selection from the data or plot context, such as unit scaling, tangent-region selection, gradient-color grouping, reference-wave detection, or result-column selection. Scientific inputs that are not inferred, such as some diffusion coefficients or formal potentials, are marked that way in the function-specific table.

Python docstrings now use concise NumPy-style summaries and point to e.describe_options('function_name') for current option tables, so option details stay in one place. Calling e.describe_options() by itself shows the valid option sections menu, starting with 'all'. Use e.describe_options('all') for every option row, e.describe_options('multiplot') for one section, or simulation tables such as e.describe_options('simulation.fit_cv'). Unknown section names print a friendly suggestion when available and show the same section menu. The helper displays tables by default; pass {'pretty print': False} for plain text output, {'print': False} to suppress output, and {'return dataframe': True} when you want the pandas DataFrame returned.

```
e.describe_options()
e.describe_options('all')
e.describe_options('multiplot')
e.describe_options('simulation.fit_cv')
e.set_defaults('print', False)
e.set_defaults('fowa', {'fit basis': 'y', 'min r2': 0.98})
e.reset_defaults_option('plot')
e.reset_defaults()
```

Canonical option-section names

Public function / section	Use with describe_options(...)
get_data	e.describe_options('get_data')
plot	e.describe_options('plot')
multiplot	e.describe_options('multiplot')
multimultiplot	e.describe_options('multimultiplot')
multi_scatterplot	e.describe_options('multi_scatterplot')
filter	e.describe_options('filter')
sort_group	e.describe_options('sort_group')
peak_current	e.describe_options('peak_current')
fit_peak_current	e.describe_options('fit_peak_current')
fit_peak_potential	e.describe_options('fit_peak_potential')
fit_rate	e.describe_options('fit_rate')
sevcik_analysis	e.describe_options('sevcik_analysis')
trumpet_analysis	e.describe_options('trumpet_analysis')
fowa	e.describe_options('fowa')
normalize_current	e.describe_options('normalize_current')
scale_current	e.describe_options('scale_current')
tafel_analysis	e.describe_options('tafel_analysis')

Public function / section	Use with describe_options(...)
normalize	e.describe_options('normalize')